I. Agenda

Datum	Inhalte	Details
02.10.2026 17:00 – 20:45 Uhr	Vorstellung der LV-Inhalte und Beurteilungskriterien, Vorbereitung Systeme und Anwendungen (Packages etc.), Vorstellung Case Study	Online, Einführung in Machine Learning
09.10.2026 13:45-17:00 Uhr	Klassifikation	**Logistische Regression**, K-nearest Neighbor, **Support Vector Machine**
10.10.2026 12:45 – 16:00 Uhr	Vorhersagemodelle	**Lineare Regression**, **K-nearest Neighbor Regression**, K-Means
16.10.2026 18:15 – 21:30 Uhr	Clustering	Hierarchisches Clustering
17.10.2026 08:00 – 12:00 Uhr	Entscheidungsbaum, Präsentation Case Study,	Regressions- und Klassifikationsbäume
23.10.2026 13:45 – 18:00 Uhr	Assoziationsregeln, Wiederholung	Online, A priori Algorithmus (Warenkorbanalyse)
04.12.2026 13:45 – 15:00 Uhr	(Hand-)schriftliche Klausur	

Die Einteilung der LV-Inhalte kann sich noch im Laufe des Semesters ändern.

Leistungsbeurteilung & Lehrinhalte

Lehrinhalte	**Grobziele**
Clusteranalyse	zu beschreiben was eine Clusteranalyse ist und wie diese angewendet wird Ergebnisse einer Clusteranalyse im Unternehmenskontext zu interpretieren und strategische Entscheidungen daraus abzuleiten
Klassifikation	zu beschreiben was eine Klassifikation ist und wie diese angewendet wird Ergebnisse einer Klassifikation im Unternehmenskontext zu interpretieren und strategische Entscheidungen daraus abzuleiten
Entscheidungsbaum	zu beschreiben was ein Entscheidungsbaum ist und wie diese angewendet wird Ergebnisse von Entscheidungsbäumen im Unternehmenskontext zu interpretieren und strategische Entscheidungen daraus abzuleiten
Assoziationsregeln	zu beschreiben was Assoziationsregeln sind und wie diese angewendet werden

	Ergebnisse von Assoziationsregeln im Unternehmenskontext zu interpretieren und strategische Entscheidungen daraus abzuleiten
Vorhersagemodelle	zu beschreiben was Vorhersagemodelle sind und wie diese angewendet werden Ergebnisse von Vorhersagemodellen im Unternehmenskontext zu interpretieren und strategische Entscheidungen daraus abzuleiten

Entnommen aus Syllabus zur LV in CAMPUS02 online

Teilleistung	**Gewichtung**
Klausurarbeit schriftlich	70% (Mindesterfordernis 50%)
Präsentation	30% (Mindesterfordernis 50%)
Für beide Teilleistungen gibt es im 3. Antritt zusätzlich ein mündliches Prüfungsgespräch.	

Verwendete Programme und Literatur

R, R Studio

Packages: tidyverse, caret, e1071, ROCR, class, knn, tree, arules, arulesViz

Burkov, A. (2019). The Hundred-Page Machine Learning Book. Vereinigte Staaten: Andriy Burkov. https://themlbook.com/ , https://themlbook.com/wiki/doku.php

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2014). An Introduction to Statistical Learning: with Applications in R.
https://www.statlearning.com/

Agrawal, R., Imieliński, T., & Swami, A. (1993). Mining association rules between sets of items in large databases. In *Proceedings of the 1993 ACM SIGMOD international conference on Management of data* (pp. 207-216).

Dalpiaz D., (2020). R for Statistical Learning. https://daviddalpiaz.github.io/r4sl/

Boehmke, B., & Greenwell, B. M. (2019). Hands-on machine learning with R. Chapman and Hall/CRC. https://bradleyboehmke.github.io/HOML/

Heesen, B. (2023). Künstliche Intelligenz und Machine Learning mit R. Springer/Gabler.
https://link.springer.com/book/10.1007/978-3-658-41576-1

Fallstudie – Titanic Survival Challenge

Die Fallstudie kann in **Gruppen von bis zu drei Personen** abgeschlossen werden und wird für alle Gruppen am gleichen Datensatz (train und test Datensatz im Moodlekurs) ausgeführt. Die Aufgabe besteht darin, vorherzusagen, welche Passagiere das Unglück auf der Titanic überleben.

Folgende Aufgabenstellungen sollen erfüllt werden:

- Beschreiben Sie die vorliegenden Daten in verbaler und visueller Form. Dazu ist es notwendig einige explorative Analyseschritte vorzunehmen.
- Untersuchen Sie den Datensatz auf ggfs. notwendige Vorverarbeitungsschritte und dokumentieren Sie diese.
- Wenden Sie mindestens drei verschiedene (müssen nicht aus der LV sein) Machine Learning Ansätze an und beschreiben Sie Ihr Vorgehen.
- Ein Ergebnis der Fallstudie ist eine Aussage darüber, welche Personen im Testdatensatz überleben und welche nicht.

Die Beurteilungsgrundlage für die Fallstudie bildet allerdings nicht die Qualität des Ergebnisses, sondern die Qualität Ihrer Präsentation. Dazu zählt:

- Sind Analyseschritte und vorangegangene Überlegungen nachvollziehbar argumentiert? z.B. Ein K-Means Clustering wurde als explorative Methode eingesetzt, um Informationen über mögliche Segmentierungen innerhalb der Passagiergruppen zu erhalten.
- Sind Ergebnisse und Güte der Modelle nachvollziehbar dokumentiert? Siehe dazu Modell Performance in den LV Unterlagen.
- Sind die Hyperparameter der Modelle nachvollziehbar gewählt?
- Sind die Ergebnisse und Erkenntnisse aus den Modellen ansprechend präsentiert?

Abzugeben sind ein zusammenfassendes Dokument mit Ihren Begründungen und Überlegungen sowie Ergebnissen (**ca. 3-5 Seiten, inkl. Screenshots**) und das jeweilige Code-File. Die Aufgaben können in einer Programmiersprache Ihrer Wahl gelöst werden.

Die Fallstudie ist bis **zum 17.10.2026 LV-Beginn** fertigzustellen und wird an diesem Tag von jeder Gruppe präsentiert. Die Präsentationszeit beträgt **maximal 15 Minuten** (anschließend ca. 10 Minuten Diskussion) und die Präsentationsform kann frei gewählt werden. Für die Beurteilung wird ausschließlich Präsentation herangezogen.

Der Titanic Datensatz ist Grundlage für zahlreiche Challenges und Tutorials, Sie haben natürlich die Möglichkeit von bereits vorangegangenen Bearbeitungen zu profitieren. Einige gelungene Versuche finden Sie hier: https://www.kaggle.com/competitions/titanic , https://www.kaggle.com/code/nikitakudriashov/top-1-titanic-solution

Zur Ausführung der Modelle und Analyseschritte können generative Modelle herangezogen werden, bitte kennzeichnen Sie jene Stellen mit einer Fußnote und dem benutzten Prompt.

II. Machine Learning Basiswissen

Machine Learning Algorithmen lassen sich grob in vier Ansätze einteilen (Burkov, 2019, S. 1 ff.):

- Supervised Learning (z.B. Regression, Klassifikation)
- Semi-supervised Learning
- Unsupervised Learning (z.B. Clustering, Dimensionsreduktion, Outlier Detection)
- Reinforcement Learning

Wichtige begriffliche Unterscheidungen im Machine Learning umfassen:

- Parameter vs. Hyperparameter
- Klassifikation vs. Regression
- Modellbasiertes vs. Instanzenbasiertes Lernen
- Shallow vs. Deep Learning

III. Klassifikation

Support Vector Machine (siehe Statistical Learning, S. 367 ff.)

SVMs (Support Vector Machines) eignen sich vor allem für Klassifikationsprobleme, werden aber auch für Regressionsprobleme und zur Ausreißer-Detektion angewandt. Die grundsätzliche Funktionsweise basiert auf der Nutzung einer Hyperebene (Hyperplane) zur Trennung von verschiedenen Klassen im Datensatz.

SVM nutzt die *Maximal Margin Hyperebene*, also jene die am weitesten von den Trainingsdatenpunkten entfernt ist und trennt anhand dieser zwischen den zugrundeliegenden Klassen. Nachdem eine scharfe Trennung der Datenpunkte meist nicht möglich ist, erlaubt der Algorithmus auch „falsch“ klassifizierte Datenpunkte im Trainingsdatensatz. Die Hyperebene und deren Margin die zur Trennung gewählt wurde, wird daher auch als *Soft Margin* bezeichnet. Der Hyperparameter C bestimmt die Anzahl der tolerierten Falschklassifikationen.

Als *Support Vektoren* werden jene Datenpunkte bezeichnet, die ausschlaggebend für die Position der Hyperebene sind.

```
Call:
svm(formula = class ~ ., data = heart01Train,
    kernel = "linear", cost = 0.01, scale = FALSE)


Parameters:
   SVM-Type:  C-classification
 SVM-Kernel:  linear
       cost:  0.01

Number of Support Vectors:  217

 ( 108 109 )


Number of Classes:  2

Levels:
 healthy risk
```

Abbildung 1 Summary Funktion eines SVM Modells in R.

In Abbildung 1 wird der Output der summary() Funktion eines SVM Modells in R dargestellt.

Logistische Regression (siehe Statistical Learning, S. 129 ff.)

Im Gegensatz zu SVM liefert die Logistische Regression als Output Wahrscheinlichkeiten (also Werte zwischen 0 und 1), die erst in Klassenzugehörigkeiten übersetzt werden müssen. Namensgebend ist die zugrundeliegende logistische Funktion:

$$p(X) = \frac{e^{\beta_0+\beta_1 X}}{1 + e^{\beta_0+\beta_1 X}}$$

Abbildung 2 zeigt die summary() Funktion eines logistischen Regressionsmodells. Dabei ist zu beachten, dass sämtliche Koeffizienten als *log Odds* dargestellt sind. Um diese besser interpretieren zu können, bietet sich eine Transformation (durch Einfügen in die Exponentialfunktion) in das sogenannte *Odds Ratio* an.

```
Call:
NULL

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -3.008203   0.970582  -3.099  0.00194 **
chol         0.002518   0.002588   0.973  0.33056
age          0.040402   0.015294   2.642  0.00825 **
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 336.61  on 243  degrees of freedom
Residual deviance: 326.90  on 241  degrees of freedom
AIC: 332.9

Number of Fisher Scoring iterations: 4
```

Abbildung 2 Summary Funktion einer Logistischen Regression in R.

Für jedes Feature ist außerdem ein p-Value und dessen Signifikanzniveau angegeben.

Neben SVM und der logistischen Regression gibt es natürlich zahlreiche, weitere Ansätze zur Lösung von Klassifikationsproblemen. Zu den bekanntesten Modellen zählen: Naïve Bayes, Linear Discriminant Analysis, K-nearest Neighbors und Entscheidungsbäume.

K-Nearest Neighbors
(siehe Statistical Learning, S. 39ff., The Hundred-Page Machine Learning Book, S. 34ff)

Der K-nearest Neighbors Classifier identifiziert jene Datenpunkte, die im Trainingsdatensatz am nächsten zum Testdatenpunkt x_0 sind und vergibt dann das am häufigsten vorkommende Label. Wie die Distanz zwischen zwei Punkten bemessen wird, hängt vom jeweiligen Distanzmaß ab. In vielen Fällen wird dabei auf die Euklidische Distanz zurückgegriffen. Wenn dies der Fall ist, sollte bei der Datenvorverarbeitung nicht auf die Skalierung der Daten verzichtet werden. Die Wurzel der summierten Euklidischen Distanz kann wie folgt dargestellt werden:

$$\sqrt{\sum_{j=1}^{P} (x_{aj} - x_{bj})^2}$$

Als Alternative dazu bietet sich die Cosinus Ähnlichkeit an. Diese misst die Ähnlichkeit der Richtung zweier Vektoren und kann daher für Vektoren unterschiedlicher Längen (vor allem im

Text Mining) verwendet werden. Weitere Metriken sind z.B. Manhattan Distanz. Neben dem Distanzmaß ist auch die Anzahl der zu betrachtenden Nachbarn (K) ein Hyperparameter für KNN.

Model Performance Assessment
(siehe The Hundred-Page Machine Learning Book, S. 13ff.)

Folgende Metriken werden am häufigsten zur Modellbewertung bei Klassifikationsproblemen herangezogen:

Confusion Matrix

Die Confusion Matrix stellt in tabellarischer Form dar, wie viele Beispiele im Datensatz korrekt klassifiziert wurden.

```
Confusion Matrix and Statistics

          Reference
Prediction  0  1
         0 25 12
         1  7 15

               Accuracy : 0.678
                 95% CI : (0.5436, 0.7938)
    No Information Rate : 0.5424
    P-Value [Acc > NIR] : 0.02391

                  Kappa : 0.3417

 Mcnemar's Test P-Value : 0.35880

            Sensitivity : 0.7812
            Specificity : 0.5556
         Pos Pred Value : 0.6757
         Neg Pred Value : 0.6818
             Prevalence : 0.5424
         Detection Rate : 0.4237
   Detection Prevalence : 0.6271
      Balanced Accuracy : 0.6684

       'Positive' Class : 0
```

Abbildung 3 Confusion Matrix SVM mit Cost = 0,01

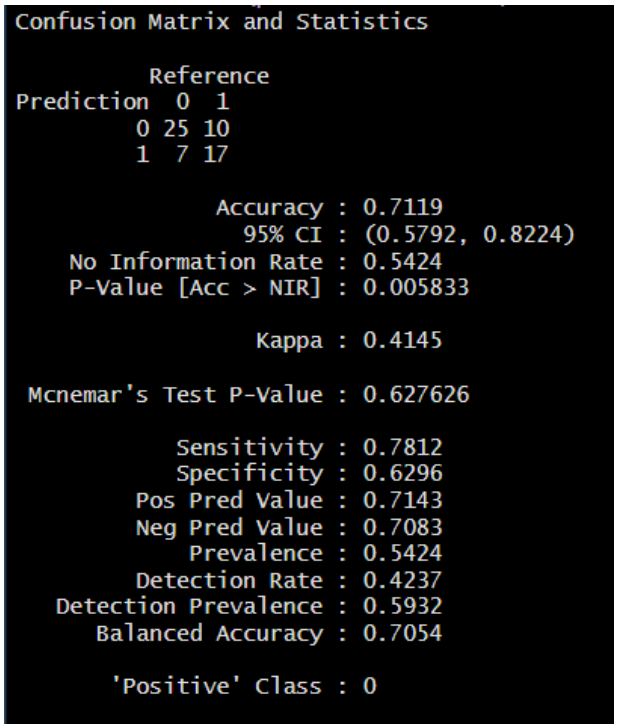


Abbildung 4 Confusion Matrix SVM mit Cost = 5.

Abbildung 3 und Abbildung 4 zeigen die Confusion Matrix und weitere Bewertungsmetriken zu einem SVM Modell, das zwischen Patient*innen, die einen Herzinfarkt erlitten haben (1) und jenen, die gesund sind (0) unterscheiden soll. Die Tabelle gleich unter der Überschrift der Abbildungen ist die eigentliche Confusion Matrix. Dabei sind *True Positives (TP)* und *True Negatives (TN)* in der Diagonale dargestellt (in Abbildung 3 25 bzw. 15). Diese Daten wurden vom Modell korrekt klassifiziert. Weitere Werte sind *False Positives* (12 in Abb. 1) und *False Negatives* (7 in Abb. 1).

Die Metrik Accuracy zeigt uns den Anteil richtig klassifizierter Datenpunkte und kann wiefolgt dargestellt werden:

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Alle übrigen Metriken besprechen wir im Rahmen der LV.

Precision/Recall

Precision gibt das Verhältnis zwischen korrekt positiven Klassifizierungen und allen als positiv klassifizierten Datenpunkten an:

$$precision = \frac{TP}{TP + FP}$$

Recall stellt die richtig positiv klassifizierten Datenpunkte, den gesamten in Wahrheit positiven Datenpunkten im Datensatz gegenüber.

$$recall = \frac{TP}{TP + FN}$$

Weitere wichtige Metriken aus der Statistik Auswertung confusionmatrix() Funktion umfassen Kappa, das Konfidenzintervall für Accuracy (CI), der p-Value und Sensitivity.

ROC-Receiver-Operating Characteristic

Die Receiver-Operating-Characteristic, ROC (siehe Abbildung 5), ist eine weitere beliebte, leicht verständliche Metrik, um Klassifikationsmodelle zu evaluieren. Dafür werden vom Modell lediglich die Wahrscheinlichkeiten der Zugehörigkeit für jeden Datenpunkt oder gleichwertige Confidence Scores benötigt. Die ROC Curve setzt Recall (True Positive Rate, TPR) und die False Positive Rate (FPR) in Relation und stellt diese graphisch dar:

$$TPR/recall = \frac{TP}{TP+FN} \text{ und } FPR = \frac{FP}{FP+TN}$$

Für alle, im Datensatz vorliegenden Threshold Werte werden Recall und FPR berechnet.

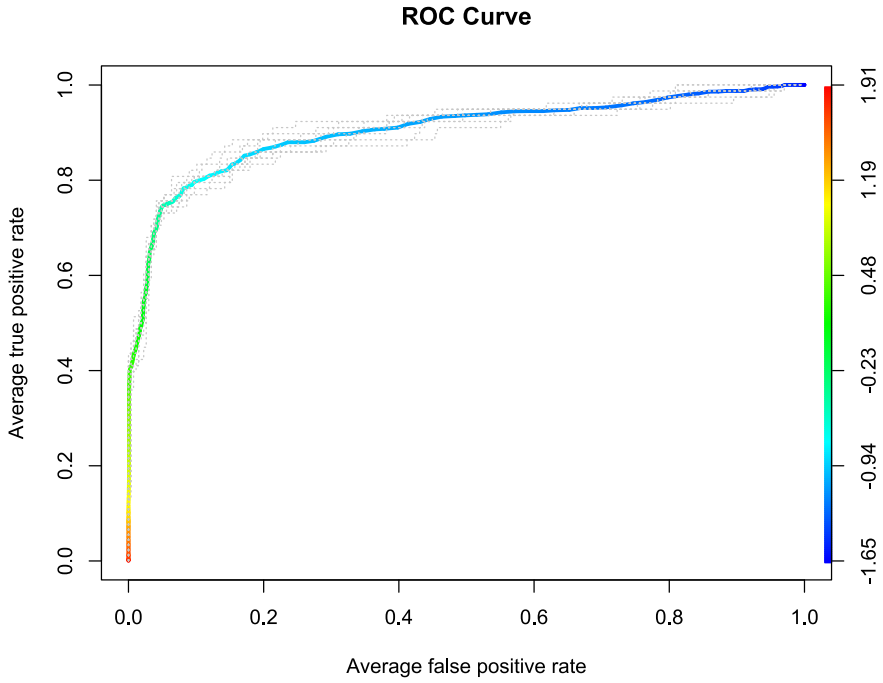


Abbildung 5 ROC Curve.

Zur Interpretation der Modellgüte wird schließlich die Fläche unter der Kurve herangezogen (area under curve, AUC). Ist diese > 0,5 kann davon ausgegangen werden, dass das Modell Ergebnisse liefert, die besser sind als eine Zufallszuordnung.

ÜBUNGEN

Übung 01 SVM

Führen Sie den SVM Algorithmus auf den gesamten heart Datensatz aus. Dabei können Sie den Code aus der LV übernehmen. Achten Sie dabei auf folgende (Ablauf-)Struktur:

1. Daten einlesen (read in data)
2. Daten analysieren und Vorverarbeitung (explore and pre-process)
3. Daten in Trainings- und Testdaten aufteilen (partition data-set)
4. Modell fitten (build model)
5. Modell auf Testdaten anwenden (run model)
6. Evaluierung (evaluation)
7. Hyperparameter tuning

Tragen Sie folgende Werte aus der Confusion Matrix in der Moodle Aktivität „Übung 01 SVM“ ein:

- Cost des besten Modells
- Accuracy
- Kappa
- Sensitivity

Übung 02 KNN

Führen Sie den KNN Algorithmus auf mindestens 4 (numerische) Variablen des heart Datensatz aus. Dabei können Sie den Code aus der LV übernehmen.
Achten Sie dabei auf folgende (Ablauf-)Struktur:

8. Daten einlesen (read in data)
9. Daten analysieren und Vorverarbeitung (explore and pre-process)
10. Daten in Trainings- und Testdaten aufteilen (partition data-set)
11. Modell fitten (build model)
12. Modell auf Testdaten anwenden (run model)
13. Evaluierung (evaluation)

Tragen Sie folgende Werte aus der Confusion Matrix in der Moodle Aktivität „Übung 02 KNN“ ein:

- Variablen
- Hyperparameter K
- Accuracy
- Kappa

Website zum Datensatz für das R-Beispiel: https://archive.ics.uci.edu/dataset/45/heart+disease

IV. Vorhersagemodelle

Lineare Regression (siehe Statistical Learning, S. 129 ff.)

Lineare Regression ist ein simpler, gut nachvollziehbarer Ansatz aus dem supervised learning um quantitative Variablen vorherzusagen. Dabei werden vor allem zwei Ziele verfolgt: Vorhersage (Hauptzweck im Machine Learning) und Inferenz. Im Rahmen unseres Vorhersageproblems suchen wir eine Funktion $\hat{f}(X)$ für die gilt:

$$Y \approx \hat{f}(X).$$

Die lineare Regression gehört zu den parametrischen Ansätzen, die das Problem der Funktionsschätzung auf das Schätzen von Parametern reduzieren. Obige Funktion nimmt mit Annahme eines linearen Modells folgende Form an:

$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \cdots + \beta_p X_p \, .$$

Um die Parameter $(\beta_0, \beta_1, \ldots \beta_p)$ auszuwählen gibt es verschiedene Ansätze, einer der häufigsten nennt sich *(ordinary) least squares (OLS)*. Dabei wird versucht eine Funktion zu finden, bei der die Summe der quadrierten Entfernungen zwischen geschätzten Punkten und tatsächlichen Datenpunkten minimiert wird (residual sum of squares, Summe der kleinsten Fehlerquadrate). Diese kann wie folgt definiert werden:

$$RSS = e_1^2 + e_2^2 + \cdots + e_n^2.$$

Als parametrischer Ansatz, ist die lineare Regression nicht sehr flexibel in ihrer Form und tendiert bei der Berücksichtigung von zu vielen Features zu *Overfitting*.

K-Nearest Neighbors Regression

K-nearest neighbors (KNN) ist ein non-parametrischer, instanzenbasierter Ansatz. Als solcher wird keine explizite Annahme über die Form der zugrundeliegenden Funktion getroffen. Wie im Abschnitt zur Klassifikation erwähnt, müssen allerdings das Distanzmaß und die Anzahl der relevanten Nachbarpunkte als Hyperparameter festgelegt werden. Während für die Klassifikation die Mehrheit der umliegenden Punkte relevant ist, wird bei der Regression der Durchschnitt der umliegenden Werte als Vorhersagewert ausgegeben.

Model Performance Assessment

$MAE\ (Mean\ absolute\ error) = \frac{1}{N}\sum_{i=1}^{N} |y_i - \hat{y}\,|.$

$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y})^2} = \sqrt{MSE}.$

- $n = Anzahl\ Beobachtungen$
- $y_i = \ der\ beobachtete\ Wert\ der\ i^{ten}\ Beobachtung$
- $\hat{y_i} = der\ vorhergesagte\ Wert\ der\ i^{ten}\ Beobachtung$
- $Root\ mean\ square\ error$

$R^2 = 1 - \frac{\sum(y_i - \hat{y})^2}{\sum(y_i - \bar{y})^2}.$

ÜBUNGEN

Übung 03 Lineare Regression / K-Nearest Neighbors Regression

Laden Sie den Datensatz *wine.rds* und führen Sie entweder eine Lineare Regression oder eine KNN Regression durch. Dabei können Sie den Code aus der LV übernehmen.
Achten Sie dabei auf folgende (Ablauf-)Struktur:

14. Daten einlesen (read in data)
15. Daten analysieren und Vorverarbeitung (explore and pre-process)
16. Daten in Trainings- und Testdaten aufteilen (partition data-set)
17. Modell fitten (build model)
18. Modell auf Testdaten anwenden (run model)
19. Evaluierung (evaluation)

Tragen Sie folgende Werte aus der Evaluierung in der Moodle Aktivität „Übung 03 Regression“ ein:

- RMSE
- R^2

Website zum Datensatz für das R-Beispiel: https://archive.ics.uci.edu/dataset/186/wine+quality

V. Clustering

Clusteringprobleme zählen zum Bereich des unüberwachten Lernens. Es gibt keine „richtige“ Lösung für unsere Aufgabenstellung und daher auch kein Test-Datenset mit dem wir unsere Ergebnisse überprüfen können. Zu den bekanntesten Clustering Algorithmen zählen K-Means und das hierarchische Clustering, DBSCAN und HDBSCAN (beide dichtebasiert).

K-Means Clustering (siehe Statistical Learning, Seite 515ff.)

K-Means ordnet alle Beobachtungen eines Datensatzes genau einem Cluster zu. Dabei muss vorab der Hyperparameter k, also die Anzahl der Cluster festgelegt werden. Innerhalb eines Clusters sollen alle Beobachtungen möglichst ähnlich sein, während sie zu Mitgliedern anderer Cluster möglichst unterschiedlich sein sollen. Die dazu herangezogene Größe wird als *within-cluster variation* bezeichnet. Um diese Größe zu definieren, benötigen wir noch ein Maß für den Abstand zwischen den Beobachtungen, dafür wird meist die Euklidische Distanz herangezogen. Wir können daraus folgendes Minimierungsproblem aufstellen:

$$\underset{C_1,\dots,C_k}{minimize}\left\{\sum_{k=1}^{K}\frac{1}{|C_k|}\sum_{i,i'\in C_k}\sum_{j=1}^{p}(x_{ij}-x_{i'j})^2\right\}$$

Der K-Means Ansatz in Schritten (siehe Abbildung 6):

1. k centroids werden zufällig platziert (Zuordnung aller Beobachtungen

2. Führe folgende Schritte aus bis sich die Clusterzugehörigkeit nicht mehr verändert:

    a. Beobachtungen werden dem naheliegendstem „centroid“ (nach Euklidischer Distanz) zugeordnet
    b. Der Mittelwert (mean) über alle Beobachtungen (Vektor der Länge p) im Cluster wird errechnet und bildet das neue „centroid“

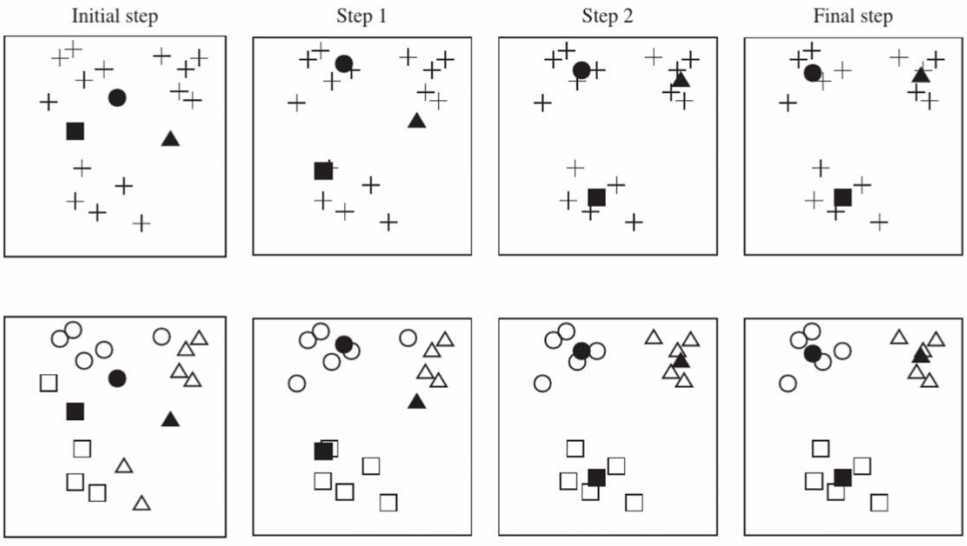


Abbildung 6 Schritte des K-Means Algorithmus nach Witten et al. (2016), S. 143.

Um den Hyperparameter *K* festzulegen, gibt es verschiedene Möglichkeiten. Ein simpler, schnell umzusetzender Ansatz ist der sogenannte *Elbow Plot (basierend auf der total within sum of squares)*. Alternativ dazu gibt es weitere Methoden, z.B. Silhouette Score, der wie folgt berechnet wird:

$$s(i) = \frac{b(i) - a(i)}{\max\{b(i), a(i)\}}$$

$a(i)$ = Durchschnittliche Distanz zwischen Datenpunkt i und allen anderen Punkten im Cluster.
$b(i)$ = Distanz zwischen Datenpunkt i und dem nächstgelegenen Cluster, zu dem i nicht gehört.

Der Silhouette Score kann Werte zwischen -1 und 1 annehmen.

Hierarchisches Clustering

Im Vergleich zu K-Means ist es beim Hierarchischen Clustering nicht notwendig den Hyperparameter K im Vorhinein zu bestimmen. Dafür muss das Distanzmaß (z.B. Euklidische Distanz) und die Art des Clusterings festgelegt werden. Resultate aus dem Ansatz werden als Baum (Dendrogramm) dargestellt. Hierarchisches Clustering kann entweder von unten – also den Blättern – nach oben (bottom-up, agglomerativ) oder von oben nach unten (top down, divisive) erfolgen. Am häufigsten wird jedoch der bottom-up Ansatz gewählt.

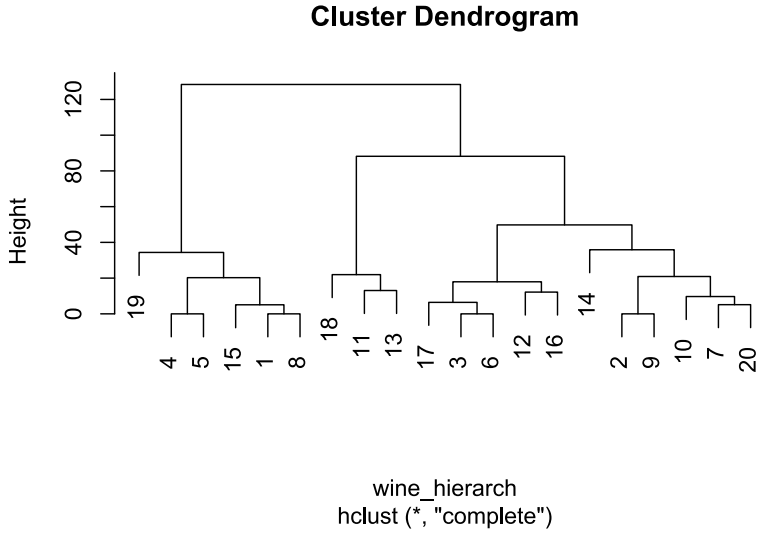


Abbildung 7 Dendrogramm, n=20, complete linkage.

Abbildung 7 stellt den Output eines hierarchischen Clusterings mit complete linkage Methode als Dendrogramm dar. Von unten gelesen finden sich zunächst die Blätter (leaves), also alle 20 Beobachtungen im Datensatz. Je näher an den Blättern sich ein Ast teilt, desto ähnlicher sind sich zwei Beobachtungen. In Abbildung 7 sind sich beispielsweise die Beobachtungen 1 und 8 ähnlicher als 8 und 15.

Der Algorithmus folgt nachstehendem Ablauf:

1. Ermittle die Distanz zwischen allen *n* Beobachtungen im Datensatz. Behandle dabei jede Beobachtung als eigenen Cluster
2. Für alle $i = n, n-1, \ldots, 2$:
    a. Untersuche alle paarweisen inter-cluster Distanzen in i Clustern und finde jenes Clusterpaar, das die geringste Distanz (also die größte Ähnlichkeit) aufweist und

verschmelze dieses Clusterpaar. Die Distanz zwischen diesen zwei Clustern entspricht der Höhe im Dendrogramm, wo sich diese beiden vereinen.

b. Berechne die neuen paarweisen inter-cluster Distanzen in den übrigen $i - 1$ Clustern.

Nachdem nicht nur Abstände zwischen zwei Beobachtungen für die Zuordnung relevant sind, sondern vor allem jene zwischen zwei Gruppen aus Beobachtungen muss hierfür noch eine passende Agglomerationsmethode ausgewählt werden. Die populärsten Methoden werden in erläutert.

Methode	Beschreibung
Complete	Maximaler inter-cluster Distanz. Berechne alle paarweisen Distanzen zwischen den Beobachtungen in Cluster A und Cluster B und schreibe die **größte Distanz** fort.
Single	Minimale inter-cluster Distanz. Berechne alle paarweisen Distanzen zwischen den Beobachtungen in Cluster A und Cluster B und schreibe **die kleinste Distanz** fort. Dies kann zu großen Clustern führen, zu denen jede Beobachtung einzeln hinzugefügt wird.
Average	Durschnittliche inter-cluster Distanz. Berechne alle paarweisen Distanzen zwischen den Beobachtungen in Cluster A und Cluster B und schreibe die **durchschnittliche Distanz** fort.
Centroid	Distanz zwischen dem Centroid für Cluster A und dem Centroid für Cluster B. Dies kann zu Inversionen führen.

Tabelle 1 Häufigste Linkage Verfahren in Anlehnung an James et al. (2014), S.524.

Herausforderungen im Clustering (siehe Statistical Learning, S. 528ff.)

Um Clusteringmethoden anwenden zu können, müssen vorab einige Hyperparameter festgelegt werden:

- Ist es sinnvoll die Daten zu standardisieren/normalisieren?
- K-Means: Wie viele Cluster K sollen gefunden werden?
- Hierarchisches Clustering:
  - Welches Distanzmaß soll angewendet werden?
  - Welcher Linkage-Typ soll angewendet werden?
  - An welcher Stelle soll das Dendrogramm gekürzt werden?

Nachdem es im unsupervised Learning keine „richtige“ Lösung gibt, lässt sich nicht immer eine genaue Aussage über die Wahl der optimalen Hyperparameter treffen.

ÜBUNGEN

Übung 04 Clusteringmethoden

In Gruppen zu 4 Personen:

1. Skizzieren Sie jeweils ein Datenset, das gut für Hierarchisches Clustering geeignet ist und eines, das nicht gut dafür geeignet ist. Begründen Sie Ihre Wahl.
2. Erläutern Sie den Unterschied zwischen Standardisierung und Normalisierung von Werten (min. 2 Sätze).
3. Diskutieren Sie Herausforderungen bei der Anwendung von K-Means in Hinblick auf die Reproduzierbarkeit und Genauigkeit des Ergebnisses. Dabei kann die Analyse der Modell Hyperparameter helfen (siehe R Code).
4. Erläutern Sie das Ziel der Principal Component Analyse. Heben Sie dabei vor allem den Unterschied zu Clusteringmethoden hervor.

Dokumentieren Sie Ihre Ergebnisse in einem Word/PDF File und laden Sie dieses anschließend in folgender Aktivität hoch: Übung 04 Clusteringmethoden

VI. ENTSCHEIDUNGSBÄUME

Methoden, die auf Baumstrukturen basieren können für Regression oder Klassifikation angewendet werden. Bäume sind leicht verständlich und gut interpretierbar, jedoch hinken sie anderen Methoden im Bereich des supervised Learning (zB General Linear Models) in Hinblick auf die Vorhersagegenauigkeit hinterher.

Regressionsbäume (siehe Statistical Learning, S. 327ff.)

Regressionsbäume nutzen den Ansatz des recursive binary splitting um die Beobachtungen im Trainingsdatensatz in verschiedene Regionen zu teilen. Sobald alle Beobachtungen einer Region zugeordnet sind, kann für eine Beobachtung aus dem Testdatensatz der Targetwert ermittelt werden. Dazu wird auch die Testbeobachtung in die jeweilige Region eingeteilt, das Target bildet den Mittelwert aus allen anderen Beobachtungen in derselben Region.

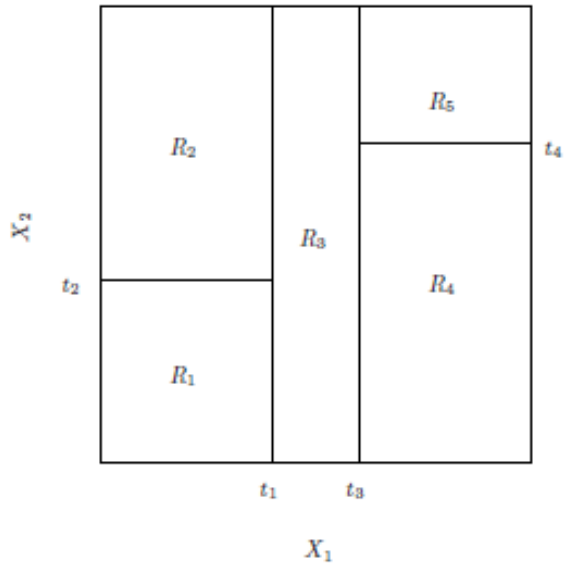


Abbildung 8 Aufteilung der Trainingswerte in verschiedene Regionen R, James et al. (2014), S. 332.

Während Abbildung 8 die Aufteilung der Trainingsbeobachtugen in die verschiedenen Regionen R darstellt, zeigt Abbildung 9 den daraus entstandenen Entscheidungsbaum.

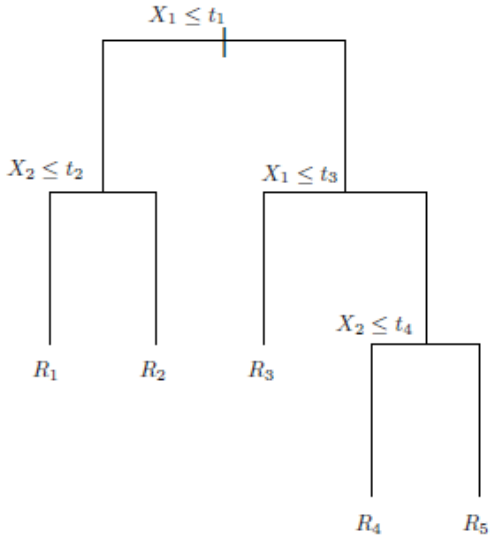


Abbildung 9 Entscheidungsbaum nach Recursive Binary Splitting, James et al. (2014), S. 332.

Um das Ergebnis zu verbessern und weniger anfällig für *Overfitting* zu machen, werden Bäume in aller Regel gekürzt (*pruning*) um einen *subtree* zu erhalten.

Klassifikationsbäume

Klassifikationsbäume liefern im Vergleich zu Regressionsbäumen diskrete Ergebnisse (Klassen). Der Ablauf zur Entstehung des Baumes gleicht dem eines Regressionsbaumen, abgesehen davon, dass der Testbeobachtung kein Mittelwert aus einer Region zugeschrieben wird, sondern der Wert der häufigsten Klasse in einer Region.

Vor- und Nachteile von Entscheidungsbäumen

+ Bäume sollen sie den menschlichen Entscheidungsprozess besser abbilden als andere Ansätze zur Regression oder Klassifikation.
+ Die graphische Darstellung von Bäumen ist übersichtlich und leicht zu interpretieren (sofern nicht zu groß).
+ Entscheidungsbäume können mit qualitativen Features (unabhängigen Variablen) umgehen und bedingen keine Umcodierung in Dummy Variablen.
– Die Vorhersagegenauigkeit von Bäumen ist meist schlechter als von anderen Modellen
– Meist sind Entscheidungsbäume nicht sehr robust, kleine Veränderungen im Datensatz können zu großen Veränderungen im Endergebnis führen.

ÜBUNGEN

Übung 05 Entscheidungsbäume

Fitten Sie einen Klassifikationsbaum auf das **heart.RDS** Datenset.

Dokumentieren Sie Ihre Ergebnisse anschließend in der zugehörigen Moodle Aktivität: Übung 05 Entscheidungsbäume

RESAMPLING

Cross-Validation (siehe Statistical Learning, S. 198ff.)

Der am Weitetesten verbreitete Ansatz von Cross-Validation ist die *k-fold Cross Validation*. Dabei wird der Trainingsdatensatz in k-Subgruppen (Folds) unterteilt. In jedem Durchlauf wird ein Fold als Validation Set ausgelassen und das Modell auf den restlichen *k-1* Folds gefittet. Die Performance wird dann am Validation Set überprüft. Die Beobachtungen sind jeweils nur einem Fold zugeordnet. Nach *k* Durchläufen wird der Mittelwert über alle Werte der Folds berechnet. Abbildung 10 stellt den Ablauf der k-fold Cross Validation schematisch dar.

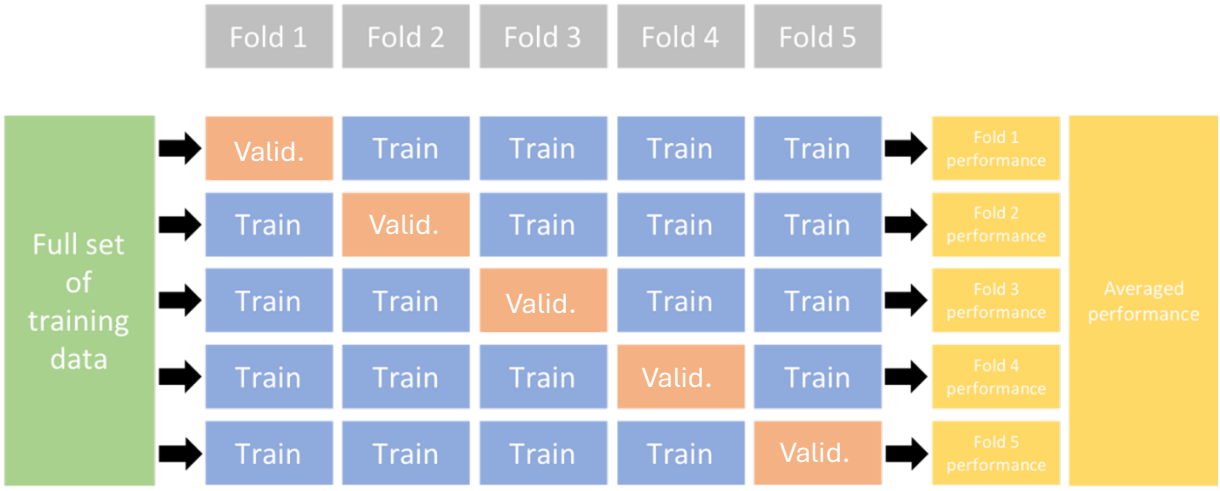


Abbildung 10 Schematische Darstellung k-fold Cross-Validation, in Anlehnung an https://bradleyboehmke.github.io/HOML/process.html

Bootstrapping

Der Bootstrap kann auf zahlreiche Methoden angewandt werden um die Unsicherheit eines Schätzers oder einer Methode zu beziffern. Aus dem Datensatz werden *n* zufällige Stichproben gezogen. Dabei können Beobachtungen in mehreren Bootstrap Samples enthalten sein (siehe z.B. blaue Beobachtungen in Abbildung 11).

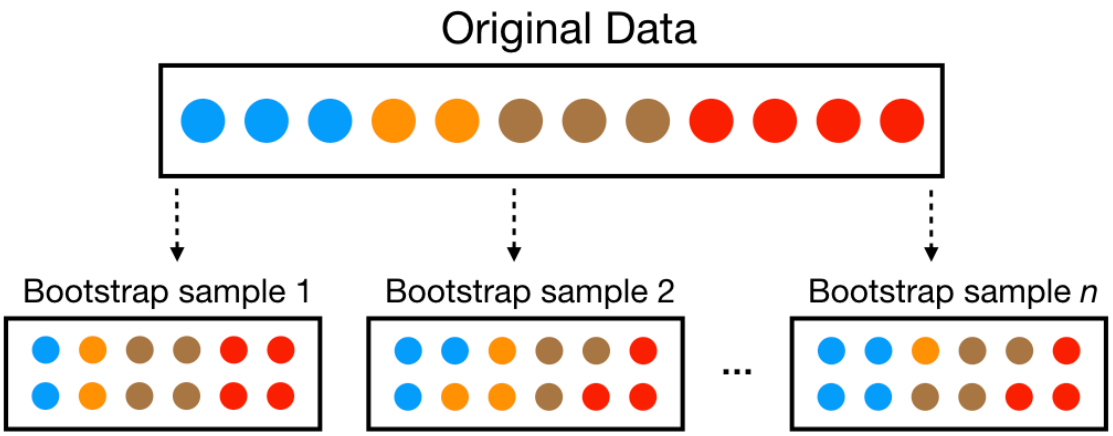


Abbildung 11 Schematische Darstellung Bootstrap aus https://bradleyboehmke.github.io/HOML/process.html

VII. Assoziationsregeln (siehe Machine Learning mit R, S. 473ff.)

Assoziationsregeln sind regelbasierte Machine Learing Modelle des unüberwachten Lernens. Ergebnis dieser Modelle sind nicht direkt Vorhersagen, sondern Zusammenhänge zwischen verschiedenen Sachverhalten. Das bekannteste Beispiel ist die Warenkorbanalyse, z.B. Welches Produkt interessiert Kundin A noch, wenn Sie bereits Produkt X gekauft hat? Unter anderem können Assoziationsregeln auch zur ABC-Klassifikation von Produkten oder Kund*innen eingesetzt werden.

Einer der bekanntesten Algorithmen in diesem Bereich ist der A priori Algorithmus (Agrawal et al. ,1993). Der Algorithmus basiert auf sogenannten Itemsets, die beispielsweise aus Warenkörben (z.B. Brot, Butter, Käse) bestehen können. Kommen gewissen Kombinationen in großer Zahl vor, werden diese vom Algorithmus als Assoziationsregel in Betracht gezogen. Dazu werden mehrere Metriken errechnet. Zunächst gibt der Support an wie wahrscheinlich es ist, dass die vorliegende Itemmenge in einer Transaktion (Einkauf) vorkommt. Die Konfidenz einer Assoziationsregel gibt deren Sicherheit an, dass die Präsenz von Item-Set X in einer Transaktion die Vorhersage erlaubt, dass auch Item-Set Y in der Transaktion enthalten sein wird. Lift gibt an, wie viel wahrscheinlicher es ist, dass auch Item-Set Y gekauft wird, wenn in einer Transaktion Item Set X gekauft wurde.

Assoziationsregel $\{Item - Set\ X\} \rightarrow \{Item - Set\ Y\}$

Eine Assoziationsregel sagt aus, dass Items auf der rechten Seite häufig mit Items aus der linken Seite gekauft wurden.

$$Support = \frac{Anzahl\ der\ Transaktionen\ mit\ X\ und\ Y}{Gesamtanzahl\ der\ Transaktionen} = P(X \cap Y)$$

$$Confidence = \frac{Anzahl\ der\ Transaktionen\ mit\ X\ und\ Y}{Gesamtanzahl\ der\ Transaktionen\ mit\ X} = \frac{P(X \cap Y)}{P(X)}$$

$$Expected\ Confidence = \frac{Anzahl\ der\ Transaktionen\ mit\ Y}{Gesamtanzahl\ der\ Transaktionen} = P\ (Y)$$

$$Lift = \frac{Confidence}{Expected\ Confidence} = \frac{P\ (X \cap Y)}{P(X) * P(Y)}$$